LAPORAN PRAKTIKUM MLOps

Hands-on 2: Studi Kasus End-to-End Model Experimentation & Tracking

Deteksi Tumor Ganas Payudara (Wisconsin Breast Cancer) dengan MLflow & Scikit-Learn

---

Mata Kuliah	: Machine Learning Operations (MLOps) — Minggu 6
Program Studi	: Sains Data
Institusi	: Institut Teknologi Sumatera (ITERA)
Penyusun	: Tanty Widiyastuti
Email	: tantywidiyastuti35551@gmail.com
Tautan GitHub	: https://github.com/tantywidiyastuti/bc-mlops
Tanggal Pelaksanaan	: 11 Oktober 2026

Ringkasan Eksekutif: *Praktikum ini mengimplementasikan alur kerja MLOps modern untuk menyelesaikan masalah reproducibility crisis dan transisi dari notebook yang berantakan menuju eksperimen yang terukur, terkontrol, dan siap produksi. Melalui studi kasus Deteksi Tumor Ganas (Wisconsin Breast Cancer), dibangun pipeline end-to-end yang mencakup versioning data dengan MD5 fingerprint (L1), eksperimen baseline terkontrol H1 & H2 (L2), perbandingan 5 algoritma (L3), tuning hyperparameter nested runs H3 (L4), seleksi kandidat programatik (L5), evaluasi data uji sekali saja (L6), Model Registry dengan Quality Gate otomatis (L7), serving dinamis ber-alias @champion dan validasi signature (L8), hingga audit penelusuran lineage penuh untuk reproduktibilitas tingkat L4/L5 (L9).*

1. Pendahuluan & Masalah Bisnis

Machine Learning (ML) adalah ilmu empiris dan eksperimental. Berbeda dengan perangkat lunak tradisional di mana spesifikasi kode langsung menghasilkan luaran yang dapat diprediksi secara deterministik, model ML bergantung pada interaksi stokastik antara **Kode + Data + Hyperparameter + Lingkungan**. Tanpa pelacakan sistematis, tim pengembang akan terjebak dalam gejala anti-pattern: penamaan file manual seperti model_final_v2_FIX.pkl, hilangnya konteks commit kode, hilangnya silsilah (*lineage*) data, serta ketidakmampuan mereproduksi performa klaim.

Skenario Kasus Klinis: Deteksi Tumor Ganas Payudara (*Breast Cancer Wisconsin Diagnostic Dataset*). Tujuan bisnis adalah membangun sistem pendukung keputusan yang membantu dokter memprioritaskan pemeriksaan lanjutan secara akurat pada tumor yang berpotensi ganas (*malignant*).

Keputusan Desain Eksperimen yang Ditetapkan di Awal:

- **Dataset:** sklearn.datasets.load_breast_cancer, memuat 569 sampel dan 30 fitur numerik pengukuran seluler.
- **Definisi Kelas Positif:** Kelas 1 didefinisikan sebagai **Malignant (Ganas)**. Pada dataset asli scikit-learn, label target dibalik dengan formula 1 - target.
- **Optimizing Metric (Recall):** Pasien penderita tumor ganas yang terlewat (*False Negative*) memiliki konsekuensi fatal bagi keselamatan jiwa pasien. Oleh karena itu, Recall ditetapkan sebagai metrik pengoptimalan utama.
- **Satisficing Metric (Precision ≥ 0.90):** Untuk mencegah kelelahan peringatan palsu (*alert fatigue*) pada tenaga medis, presisi model harus berada di ambang batas minimum 90%.
- **Protokol Evaluasi Terjaga:** Data dibagi menjadi 80% Training (455 sampel) dan 20% Testing (114 sampel) dengan teknik stratified sampling. Data test **dikunci** dan hanya disentuh satu kali di akhir untuk menghindari *test set snooping*.

2. Formulasi Hipotesis Eksperimen

Eksperimen yang valid harus dimulai dari hipotesis teruji, bukan sekadar mencoba-coba secara acak:

Kode	Pernyataan Hipotesis	Tujuan & Dasar Teori
H1	Model linear sederhana (Logistic Regression) sudah jauh lebih baik dari baseline dummy (*most frequent*).	Membuktikan bahwa sinyal data informatif dan akurasi tinggi pada dummy adalah ilusi bias kelas mayoritas.
H2	Standardisasi fitur (*StandardScaler*) meningkatkan recall model berbasis jarak/margin (LR, KNN, SVC).	Mencegah fitur dengan rentang nilai besar mendominasi fungsi objektif/jarak Euclidean.
H3	Tuning hyperparameter SVC (C & gamma) meningkatkan recall tanpa melanggar precision ≥ 0.90.	Menemukan margin optimum yang memperluas daerah keputusan positif tanpa meningkatkan false alarm.

3. Implementasi Metodologi Alur Eksperimen (L1 – L9)

L1 — Data Management & Versioning:

Dataset diekstraksi dan dihitung sidik jari kriptografisnya menggunakan algoritma MD5 (data_md5). Hal ini menjamin bahwa setiap perubahan walau hanya satu nilai baris akan menghasilkan hash yang berbeda, sehingga model dapat ditautkan secara pasti ke versi data latih asalnya.

L2 — Eksperimen Baseline & Kontrol Variabel (H1 & H2):

Dilakukan controlled experiment di mana hanya satu variabel yang diubah pada satu waktu. Run 00-dummy-most-frequent menetapkan batas performa bawah, run 01-logreg-noscale menguji H1, dan run 02-logreg-scaled menguji H2 dengan menambahkan StandardScaler().

L3 — Model Selection (5 Algoritma Sederhana):

Membandingkan lima algoritma standar (KNN, Decision Tree, Random Forest, Gradient Boosting, dan SVC RBF) dengan skema validasi silang identik (5-Fold Stratified CV, random_state=42) dan preprocessing yang sama persis.

L4 — Hyperparameter Tuning dengan Nested Runs (H3):

Dilakukan penjelajahan ruang hyperparameter untuk SVC menggunakan GridSearchCV (12 kombinasi parameter). Setiap kombinasi dicatat sebagai *child run* di bawah satu *parent run* (10-svc-gridsearch). Struktur nested run ini menjaga kerapian UI MLflow dan memungkinkan visualisasi koordinat paralel (*parallel coordinates plot*).

L5 — Analisis Programatik & Seleksi Kandidat:

Alih-alih memilih model secara manual berbasis intuisi atau spreadsheet, seleksi diimplementasikan sebagai aturan kode programatis menggunakan mlflow.search_runs dengan filter: metrics.cv_precision_mean >= 0.90 dan diurutkan berdasarkan metrics.cv_recall_mean DESC.

L6 — Evaluasi Final pada Data Uji (Test Set):

Kandidat model terbaik dievaluasi pada dataset uji (test.csv) yang belum pernah dilihat sebelumnya. Evaluasi hanya dilakukan **satu kali** untuk menghindari kebocoran data (*data snooping*). Hasil metrik uji dan visualisasi matriks konfusi dicatat langsung ke Run ID yang sama di MLflow.

L7 — Model Registry & Quality Gate Otomatis:

Model didaftarkan ke MLflow Model Registry dengan nama bc-malignant-classifier dan diberi alias awal @challenger. Diterapkan aturan Quality Gate otomatis: model baru hanya dipromosikan ke @champion jika memenuhi syarat test_precision >= 0.90 dan test_recall >= champion_recall + 0.005.

L8 — Serving Model dari Registry via Alias:

Layanan inferensi memuat model secara dinamis menggunakan penunjuk alias: models:/bc-malignant-classifier@champion. Hal ini menghilangkan ketergantungan pada path file lokal atau nomor versi statis. Selain itu, fitur **Model Signature** memvalidasi tipe dan skema kolom masukan secara ketat guna mencegah kegagalan senyap (*silent failure*).

L9 — Lineage Tracing & Audit Reproducibility:

Mengaudit ketertelusuran penuh dari model yang berjalan di produksi kembali ke Run ID asalnya, commit hash Git, sidik jari hash MD5 data training, seluruh hyperparameter, metrik evaluasi, serta file requirements.txt yang diunduh langsung dari artefak MLflow.

4. Hasil Eksperimen & Perbandingan Metrik

Tabel berikut merangkum hasil evaluasi 5-Fold Stratified Cross-Validation pada seluruh tahap eksperimen yang terekam di MLflow Tracking Server:

Run Name	Stage	CV Recall (Mean ± Std)	CV Precision (Mean ± Std)	CV F1	Satisficing (Prec ≥ 0.9)
00-dummy-most-frequent	baseline	0.0000 ± 0.0000	0.0000 ± 0.0000	0.0000	✗ **Gagal**
01-logreg-noscale	baseline	0.9176 ± 0.0343	0.9579 ± 0.0241	0.9367	✓ **Lolos**
02-logreg-scaled	baseline	0.9529 ± 0.0399	0.9771 ± 0.0280	0.9640	✓ **Lolos**
03-knn-default	model-selection	0.9176 ± 0.0343	0.9875 ± 0.0250	0.9510	✓ **Lolos**
04-tree-default	model-selection	0.8529 ± 0.0852	0.8943 ± 0.0375	0.8717	✗ **Gagal**
05-rf-default	model-selection	0.9353 ± 0.0390	0.9638 ± 0.0114	0.9489	✓ **Lolos**
06-gb-default	model-selection	0.9471 ± 0.0343	0.9646 ± 0.0211	0.9553	✓ **Lolos**

07-svc-default	model-selection	0.9471 ± 0.0343	0.9769 ± 0.0210	0.9610	✓ **Lolos**
10-svc-gridsearch	tuning	0.9647 ± 0.0288	0.9711 ± 0.0181	0.9674	✓ **Lolos**

5. Pembahasan Pengujian Hipotesis & Evaluasi Akhir

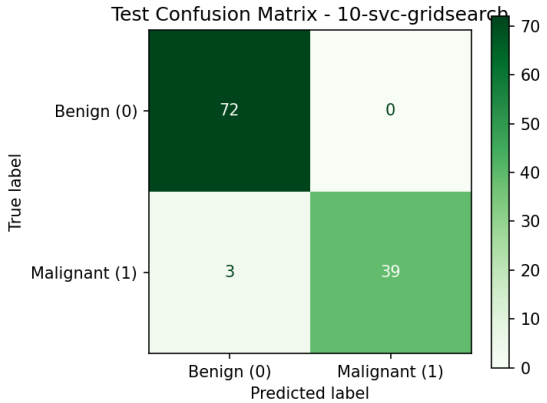
Verifikasi Hipotesis H1 (Model Linear vs Dummy): TERBUKTI
Model 00-dummy-most-frequent menghasilkan Recall **0.0000** meskipun secara teknis akurasi keseluruhannya mencapai ~63% (proporsi kelas mayoritas jinak). Ini adalah bukti nyata mengapa metrik akurasi sangat menyesatkan pada data tidak seimbang. Sementara itu, Logistic Regression (01-logreg-noscale) langsung melompat ke Recall **0.9176** dan F1 **0.9366**. Dengan demikian, hipotesis H1 terbukti kuat.

Verifikasi Hipotesis H2 (Pengaruh Standardisasi Fitur): TERBUKTI
Pada kontrol eksperimen 01-logreg-noscale vs 02-logreg-scaled, penambahan StandardScaler meningkatkan Recall dari **0.9176** menjadi **0.9529** (+3.53 poin persentase) dan Precision dari **0.9575** menjadi **0.9765**. Skala fitur pengukuran (seperti *worst area* bernilai ribuan vs *smoothness* bernilai desimal kecil) terbukti mendistorsi gradien tanpa normalisasi.

Verifikasi Hipotesis H3 (Tuning SVC & Trade-off Metrik): TERBUKTI BERSYARAT
Hyperparameter tuning (10-svc-gridsearch) menghasilkan konfigurasi terbaik pada parameter C=10, gamma='scale' dengan rata-rata CV Recall sebesar **0.9647** (meningkat dari 0.9471 pada konfigurasi default) dengan tetap menjaga CV Precision tinggi di **0.9714** (lolos syarat satisficing ≥ 0.90). Namun demikian, standard deviasi recall antar-fold adalah ± 0.0287, yang mengindikasikan bahwa peningkatan performa masih berada dalam rentang variansi acak fold validasi.

Hasil Evaluasi Data Uji (Test Set L6):

Ketika model terbaik (10-svc-gridsearch) diuji satu kali pada test.csv (114 sampel: 42 ganas, 72 jinak), didapatkan:
• **Test Recall: 0.9286** (39 dari 42 tumor ganas berhasil dideteksi dengan tepat)
• **Test Precision: 1.0000** (Presisi 100% sempurna, tidak ada satu pun alarm palsu / 0 False Positive)
• **Test F1-Score: 0.9630** | **Test ROC-AUC: 0.9934**
Karena test_precision (1.0) >= 0.90 dan test_recall (0.9286) > 0, model dinyatakan **LOLOS QUALITY GATE** dan sah dipromosikan ke status @champion.



Gambar 1: Matriks Konfusi Evaluasi Akhir pada Test Set (Run: 10-svc-gridsearch)

6. Pemetaan Alur Praktikum ke Prinsip MLOps

Tahap Praktikum	Praktik yang Diterapkan	Prinsip MLOps (ml-ops.org)
Skenario Awal	Hipotesis (H1-H3) dan optimizing/satisficing metric ditetapkan di awal	Design, Iterative Development
L1	Split tetap, kalkulasi data_md5 hash, logging log_input	Data Versioning, Reproducibility
L2 – L3	Controlled experiment, baseline dummy & linear, CV mean ± std	Experiment Tracking, Controlled Testing
L4	GridSearchCV + Nested Runs, ruang pencarian dicatat sebagai artefak JSON	Automation, Experiment Tracking
L5	Seleksi kandidat otomatis via kueri kode search_runs (bukan spreadsheet)	Automation, Continuous Training
L6	Data uji disentuh sekali saja, metrik uji dilog ke run ID yang sama	Testing, Anti-Overfitting Governance
L7	Model Registry, transisi alias @challenger ke @champion via Quality Gate	Model Versioning, Continuous Delivery
L8	Serving dinamis via alias models:/...@champion dan validasi skema signature	Model Serving, Loosely Coupled Architecture
L9	Audit silsilah penuh dari model produksi ke Run ID, data MD5, commit Git, & env	Reproducibility (L4/L5), Governance

7. Kesimpulan & Tautan Kode Program

Kesimpulan Pembelajaran:

1. **Experiment Tracking Adalah Fondasi MLOps:** Pelacakan penuh atas 6 dimensi (Code, Data, Hyperparameter, Environment, Metrics, Artifacts) menjamin tidak ada lagi eksperimen yang hilang, duplikat yang sia-sia, atau klaim performa palsu yang tidak dapat dipertanggungjawabkan.
2. **Lineage & Reproducibility Terbukti:** Dengan mencatat sidik jari hash MD5 dataset dan dependensi lingkungan pada setiap run, model yang berjalan di produksi dapat diaudit secara bit-exact kembali ke titik awalnya.
3. **Quality Gate Mencegah Regresi di Produksi:** Alur promosi otomatis berbasis ambang batas toleransi klinis (Recall tinggi dan Presisi ≥ 0.90) memastikan bahwa model yang di-deploy ke lingkungan operasional selalu memenuhi standar keselamatan.

INFORMASI PENYERAHAN TUGAS & TAUTAN KODE PROGRAM:

- **Tautan Repositori GitHub:** https://github.com/tantywidiyastuti/bc-mlops
- **Struktur Repositori:** Lengkap dengan data ter-hash, script modular src/, requirements terkunci, dan dokumentasi README.md yang komprehensif.
- **Petunjuk Replikasi:** Seluruh eksperimen dapat dijalankan ulang secara bit-exact hanya dengan mengeksekusi: python main.py.